Recipe 2: Creating Custom Job Classes

Now, let's say you're dealing with a situation where certain business processes—like generating comprehensive reports or processing large datasets—need to run asynchronously to avoid messing with the user experience. Your team decided that moving these tasks to the background would make operations more efficient and improve responsiveness. To do this, you decided to put different business logic into custom job classes that would run asynchronously. This approach makes sure that your main application stays responsive while heavy or time-consuming tasks run in the background. Now, we will learn how to create and configure custom job classes for asynchronous processing using Laravel's job system, optimize task automation, and maintain efficiency.

Creating Custom Job Class

Generating Job Class

First, we need to start by generating a custom job class with an Artisan command. And for this, simply run the following: